

Utility di compressione di immagini tramite DCT

Benchmark e comparatore della compressione DCT2

Nicolas Chines (matr. 899536)
Jacopo Borgato (matr. 866305)

A.A.: 2025/2026

Sommario

Il progetto consiste nella realizzazione di due software che implementano la compressione delle immagini basata su DCT.

Nella prima parte viene implementato l'algoritmo DCT2, confrontando successivamente i tempi di esecuzione con quello implementato dalla libreria numpy.

Per la seconda parte il programma permette all'utente di scegliere da filesystem un'immagine bitmap in toni di grigio, applicare la compressione secondo i parametri specificati e visualizzare affiancate l'immagine originale e compressa.

1 Introduzione

La funzione `dct2` calcola la trasformata discreta del coseno 2D di una matrice 2D, tipicamente un'immagine. Questa trasformata converte i dati spaziali in componenti di frequenza. È una tecnica fondamentale per la compressione di immagini, alla base dello standard JPEG.

L'operazione è separabile: equivale a applicare una DCT 1D prima sulle righe e poi sulle colonne (o viceversa). I coefficienti ottenuti permettono di ricostruire l'immagine originale tramite la trasformata inversa (`idct2`). Durante la compressione, i coefficienti ad alta frequenza, che risultano essere meno percepibili all'occhio umano, possono essere eliminati per ridurre la dimensione del file con una perdita di qualità limitata.

2 Implementazione DCT / DCTN

L'idea che sta alla base del calcolo della DCT è la trasformazione da base canonica in uno spazio vettoriale a quella dei coseni.

2.1 Implementazione DCT1

Come fattore di normalizzazione viene usato $1/\sqrt{N}$ per il primo coefficiente, per i restanti invece $\sqrt{2/N}$.

In fase di inizializzazione viene creato un vettore di output `y`, di lunghezza pari alla dimensione del vettore, pieno di zeri.

```
1 N = len(v)
2 y = np.zeros(N, dtype=float)
```

Successivamente viene calcolato il coefficiente `k` per ogni posizione nel seguente modo:

```
1 if k == 0:
2     Ck = 1.0 / math.sqrt(N)
3 else:
4     Ck = math.sqrt(2.0 / N)
5
6 for i in range(N):
7     ak = math.cos((math.pi * k * (2*i + 1)) / (2 * N)) * v[i]
8     sum_cos += ak
9     y[k] = Ck * sum_cos
```

Dove `Ck` è il fattore di normalizzazione, `ak` è la base della DCT, che rappresenta le diverse frequenze, la quale viene aggiunta alla somma dei coseni.

In questo modo non si ha più informazione locale come nella base canonica ma si ha informazione riguardante tutto il vettore.

Il risultato è quindi il vettore `y` di coefficienti che rappresenta l'input in termini di componenti di frequenza del coseno.

2.2 Implementazione DCT2

Per calcolare la funzione DCT2 é possibile iterare su righe e colonne la DCT1.

```
1  # DCT1 for columns
2  for j in range(N):
3      y_m[:,j] = dct(m_in[:,j])
4
5  # DCT1 for rows
6  for j in range(N):
7      y_m[j,:] = np.transpose(dct(np.transpose(m_in[j,:])))
```

Il primo passaggio è scorrere ogni colonna della matrice originale applicando la funzione `dct1`, salvando il risultato nella corrispondente colonna.

Successivamente si applica lo stesso passaggio, alla matrice ottenuta precedentemente, ma sulle righe.

3 Benchmark

Il file `benchmark.py` implementa un semplice benchmark sulla funzione `dct2` implementata, su matrici di dimensione $N \times N$. con N che ha range da 10 a 200 in step da 10. Confrontandola con quella della libreria Python `numpy`

```
n_values = np.arange(10, 201, 10)
```

Per poi mostrare i risultati a schermo tramite la libreria `matplotlib`.

```
1  for n in n_values:
2      x = np.random.randint(0, 255, (n, n)).astype(float) -
          128
3
4      start_t = time.perf_counter()
5      _ = dctn(x, type=2, norm='ortho')
6      end_t = time.perf_counter()
7      scipy_times.append((end_t - start_t) * 1000) #
          Convertiamo in ms
8
9      start_t = time.perf_counter()
10     _ = utils.dct2(x)
11     end_t = time.perf_counter()
12     custm_times.append((end_t - start_t) * 1000) #
          Convertiamo in ms
13
14     print(f"Test completato per N={n}")
15
16  return np.array(scipy_times), np.array(custm_times)
```

La funzione esegue quindi i seguenti step per ogni valore nel range definito:

1. Genera una matrice quadrata $n \times n$ di valori interi casuali tra 0 e 255 (simulando pixel di un'immagine)

2. Viene applicata la funzione `dctn` della libreria, tenendo traccia del tempo di esecuzione, usando i parametri `norm='ortho'` che applica una normalizzazione ortonormale. e `type=2` che utilizza lo standard per la compressione.
3. Viene misurato il tempo della funzione `dct2` implementata non dalla libreria sulla stessa matrice
4. Infine vengono salvati i tempi di esecuzione per essere successivamente mostrati